Znakovni nizovi

Kako pronaći bilo koji komadić teksta, i to brzo?

dr.sc.tech. Kristijan Poje, 2026.

Leo i citat koji kasni

- Leo je duša društva: uvijek ima pravi citat iz filma ili knjige — u pravom trenutku.
- Problem: pamti samo *komadiće*: "drizzly November", "universally acknowledged".
- Citat koji stigne minutu prekasno ne vrijedi ništa.
- Mobitel ga spasi: pretraži cijeli *Moby-Dick* (~1,2 MB) — ali svaki put **čita cijelu knjigu ispočetka**.

Tri pitanja koja postavlja svaki tekstni indeks

Tekst od *n* znakova, uzorak (komadić) od *m* znakova. Svaka struktura danas odgovara na ista tri pitanja:

- `count` — pojavljuje li se uzorak i koliko puta?
- `locate` — na kojim pozicijama?
- `extract` — daj mi tekst oko te pozicije (cijeli citat).

Primjene: provjera plagijata, pretraživanje koda (GitHub), poravnavanje DNA očitanja na genom od 3 milijarde slova, "je li jezični model ovo prepisao iz podataka za učenje?".

Gradivni blok: ista struktura podataka, različita veličina

Veličina indeksa u odnosu na tekst:

1. Sufiksni trie / sufiksno stablo — **~35×** (plus tekst)
2. Sufiksno polje (SA) — **~5×** (tekst + 4 bajta po znaku)
3. Burrows–Wheelerova transformacija (BWT) — **1×** (samo permutacija teksta, nije indeks)
4. FM-indeks — **~0,6×** (manji od knjige, a knjigu više ne trebamo)

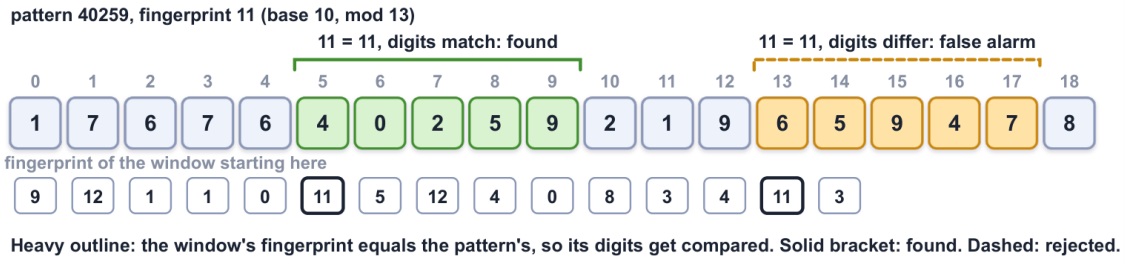
Odjeljak 1 — Bez indeksa: čitanje cijele knjige

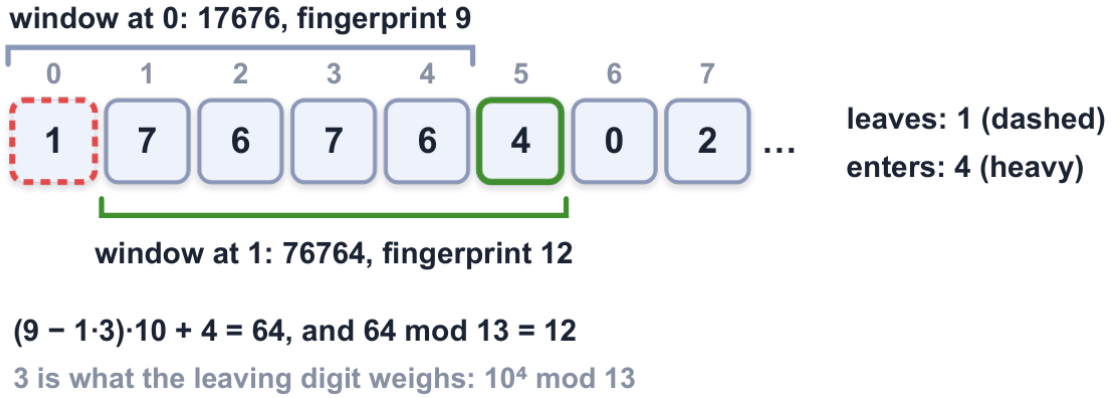
Naivno pretraživanje: klizni prozor

- Postavi uzorak na poziciju 0, uspoređuj znak po znak; prva razlika → pomakni za jedno mjesto.
- Svako postavljanje uzorka je **prozor** teksta.
- Najgori slučaj: ~$n \times m$ usporedbi. Uzorak od 20 znakova na deset knjiga → više od 100 milijuna.
- Rasipanje: susjedni prozori dijele sve znakove osim jednoga, a mi to bacamo.

Rabin–Karp i zašto ni to nije dovoljno

- Ideja: svakom prozoru dodijeli **otisak** (broj mod velik prost broj); klizanje = 3 aritmetičke operacije.
- Isti otisak znači samo "možda" → uvijek provjeri znak po znak (lažna uzbuna se odbaci).
- Očekivano ~*n* + *m* koraka. Za mnogo upita: i dalje **jedan prolaz kroz cijeli tekst po upitu**.
- GitHub: 115 TB koda, ~96 s po upitu na 2048 jezgri → "moramo izgraditi indeks".

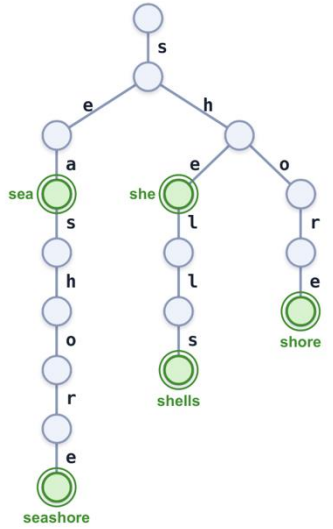




Odjeljak 2 — Trie: kad znam kako počinje

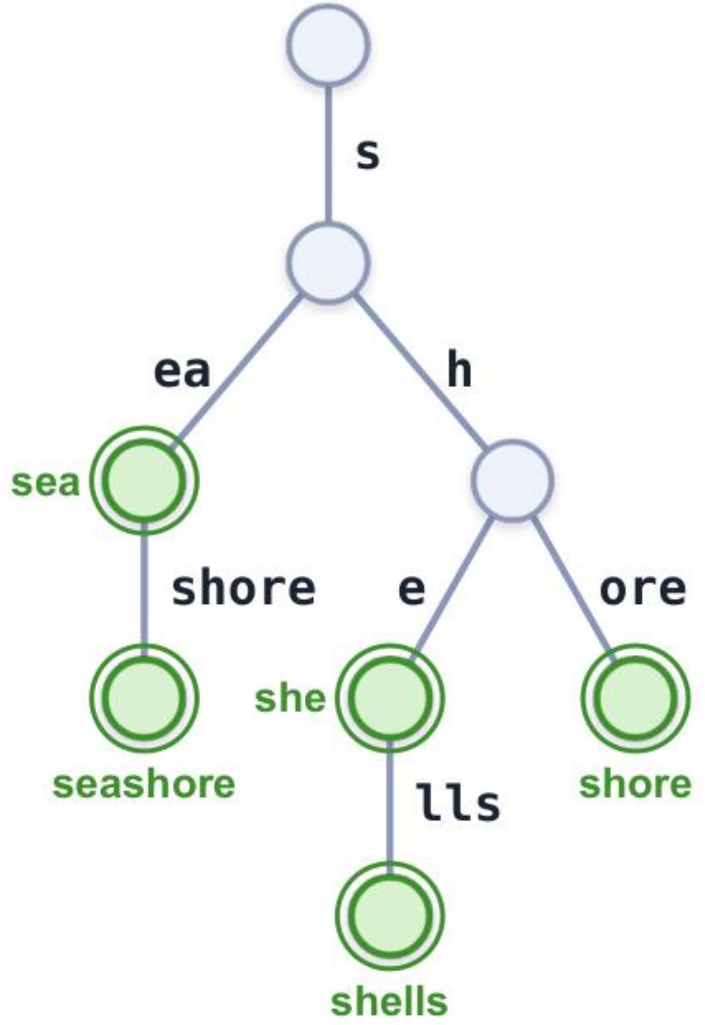
Trie: riječ je put

- Leo se sjeća početka: "It is a truth…" — kao kontakti na mobitelu: lista se sužava sa svakim slovom.
- Zajedničke početke spremi **jednom**: slova su na granama, riječ je put od korijena do oznake "ovdje završava riječ".
- Pretraga riječi od *m* slova = *m* koraka, **bez obzira na to koliko je riječi spremljeno**.
- Svi pojmovi s prefiksom: spusti se po prefiksu, pročitaj cijelo podstablo (dolaze sortirani).



Patricia stablo: stisni lance

- Problem: dugi lanci čvorova s jednim djetetom (`seashore` → s, h, o, r, e).
- U brzoj izvedbi čvor ima 256 pokazivača × 8 B ≈ 2 KB → 17 čvorova ≈ 35 KB za 25 bajtova riječi.
- Rješenje: spoji lanac u jednu granu s više slova → **Patricia stablo** (radix stablo).
- Invarijanta: svaki čvor ispod korijena završava riječ ili se grana → najviše **2k čvorova** za *k* riječi.



Što trie ne može: sredina

- Trie odgovara samo na pitanje "što **počinje** ovako?".
- Leo se najčešće sjeća sredine: "…universally acknowledged…".
- Svaki put u trieu počinje prvim slovom spremljene riječi → za sredinu nemamo ništa.

Odjeljak 3 — Od sufiksnog triea do sufiksnog polja

Ključni trik: svaki komadić je početak nekog sufiksa

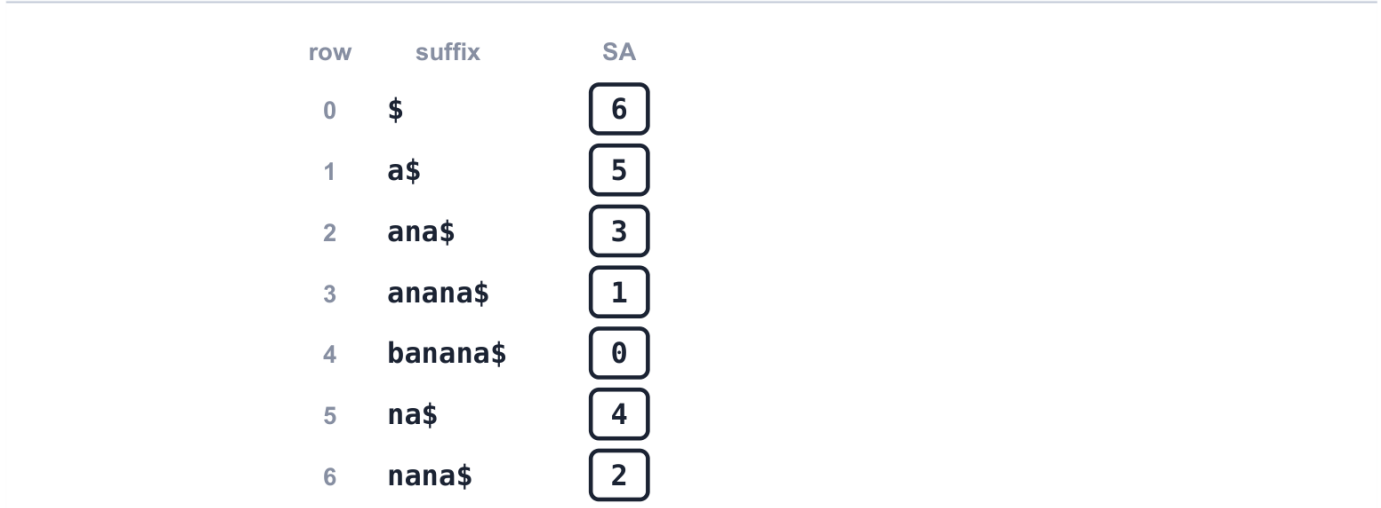
- **Sufiks** = rep teksta od neke pozicije do kraja.
- ana se pojavljuje na poziciji 1 u banana ⇔ sufiks anana **počinje** s ana.
- Dakle: "nađi komadić bilo gdje" = "nađi sufiks koji počinje tim komadićem" → to trie zna!
- Na kraj teksta dodajemo **$** (graničnik): pojavljuje se jednom i manji je od svih slova.

Sufiksni trie i sufiksno stablo: radi, ali ne stane

- Svi sufiksi u jedan trie → spusti se po uzorku, sve ispod su pojavljivanja.
- Sufiksi proze dijele malo slova → gotovo svaki sufiks ima svoj lanac: ~$n^2/2$ čvorova (za *Moby-Dick* > 700 milijardi).
- Sažeti lanci (Patricia) → **sufiksno stablo**: ~1,4 čvora po slovu, ~24 B po čvoru → **~34–35 bajtova po bajtu teksta**, plus sam tekst

Sufiksno polje: čuvaj gdje počinje, a ne što piše

- Sortiraj sve sufikse; za svaki spremi **samo početnu poziciju**.
- `banana$` → SA = `6 5 3 1 0 4 2` — sedam brojeva za sedam znakova.
- Invarijanta 1: svaka pozicija se pojavljuje točno jednom → nijedno pojavljivanje ne može promaknuti.
- Invarijanta 2: sufiksi su sortirani → sva pojavljivanja istog uzorka leže **jedno do drugoga**.
- U Pythonu: `sorted(range(n), key=lambda i: text[i:])`.

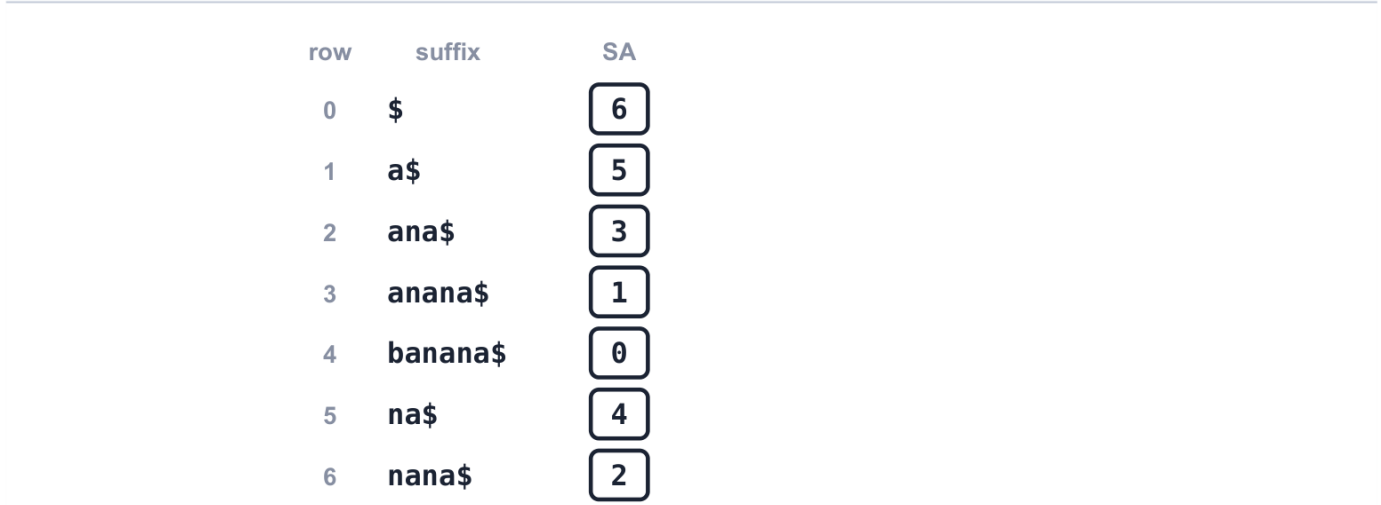


row	suffix	SA
0	$	6
1	a$	5
2	ana$	3
3	anana$	1
4	banana$	0
5	na$	4
6	nana$	2

Pretraživanje SA: binarno pretraživanje po sufiksima

Traži ana u banana$:

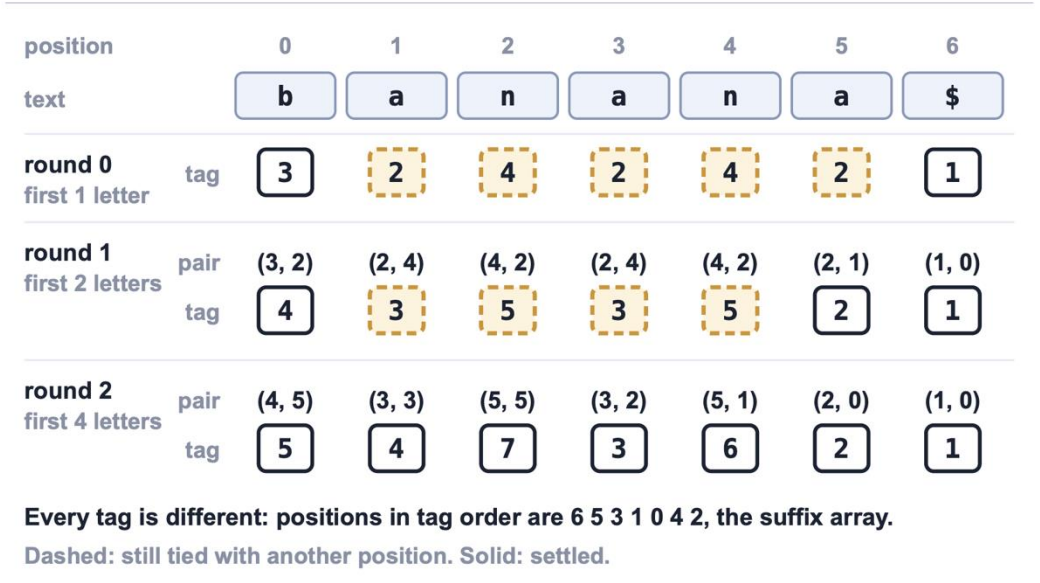
- Tražimo **gornju granicu**: prvi redak čija prva 3 slova *nisu prije* ana → retci 3, 1, 2 → **redak 2**.
- Tražimo **donju granicu**: prvi redak čija prva 3 slova *dolaze poslije* ana → retci 3, 5, 4 → **redak 4**.
- Raspon [2, 4) → retci 2 i 3 → pozicije **3 i 1**.
- Trošak: ~2 · $\log_2 n$ pogleda, svaki do m usporedbi → ~$2m \log_2 n$.



row	suffix	SA
0	$	6
1	a$	5
2	ana$	3
3	anana$	1
4	banana$	0
5	na$	4
6	nana$	2

Kako izgraditi SA: udvostručavanje prefiksa

- Obično sortiranje sufiksa je sporo kad tekst ima duga ponavljanja (milijun slova a).
- Ideja (kao kod otisaka): ne upoređuj slova, upoređuj **male brojeve (oznake)**.
- Runda 0: oznaka = prvo slovo. Runda *k*: par (oznaka(*i*), oznaka(*i* + *w*)) → oznaka za prvih **2w** slova.
- Svaka runda udvostručuje pokriveni prefiks → najviše $\log_2 n$ rundi, ukupno ~$n \log n$.
- Na deset knjiga: 8 rundi (najdulje ponavljanje 145 B), a ne 23.



LCP polje (opcionalno)

- LCP[*i*] = koliko početnih slova sufiks u retku *i* dijeli s retkom iznad.
- banana$ → LCP = `0 0 1 3 0 0 2`.
- Najveći broj u LCP = **najdulji ponovljeni odlomak** (u banana$: ana).
- Deset knjiga: 145 B — refren pjesme Lažne kornjače iz *Alise u zemlji čudesa*.
- Računa se u linearnom vremenu (Kasaijev algoritam).

row	suffix	SA	LCP
0	$	6	0
1	a$	5	0
2	ana$	3	1
3	anana$	1	3
4	banana$	0	0
5	na$	4	0
6	nana$	2	2

Koliko je velik SA?

- Tekst: *n* bajtova. SA: 4 bajta po znaku (do ~4,29 milijardi znakova).
- Ukupno **5n** — pet puta veće od knjige (deset knjiga: ~34 MB za 6,8 MB teksta). S LCP: 9n.
- Druga prečka ljestava: **35× → 5×**.
- Primjer iz prakse: SA nad 5 bilijuna tokena podataka za učenje jezičnog modela (2024.).
- Ali: tekst se i dalje mora čuvati — svaka usporedba ga čita.

Odjeljak 4 — Burrows–Wheelerova transformacija

Korisan nered? Zadatak s olovkom

- Napiši svih 7 **rotacija** od banana$: banana$, anana$b, nana$ba, …
- Sortiraj ih ($ je najmanji).
- Zadrži samo **zadnji znak** svakog retka → annb$aa.
- Pitanje: imaš samo annb$aa. Možeš li vratiti banana$?

Stupci F i L

- **F** (prvi stupac) = slova teksta sortirana: `$aaabnn`.
- **L** (zadnji stupac) = transformacija: `annb$aa`.
- Zadnje slovo retka u tekstu dolazi **neposredno prije** prvog slova tog retka.
- Rotacije odrezane na $ = sufiksi, u istom poretku → redak *i* je redak *i* sufiksnog polja.
- Zato: `L[i] = text[sa[i] - 1]` — jedan prolaz kroz SA.

row	rotation	SA	LCP	F	L
0	**$**banana	6	0	$	a_0
1	**a$**banan	5	0	a	n_0
2	**ana$**ban	3	1	a	n_1
3	**anana$**b	1	3	a	b_0
4	**banana$**	0	0	b	$\$_0$
5	**na$**bana	4	0	n	a_1
6	**nana$**ba	2	2	n	a_2

Ključno svojstvo: jednaka slova čuvaju poredak

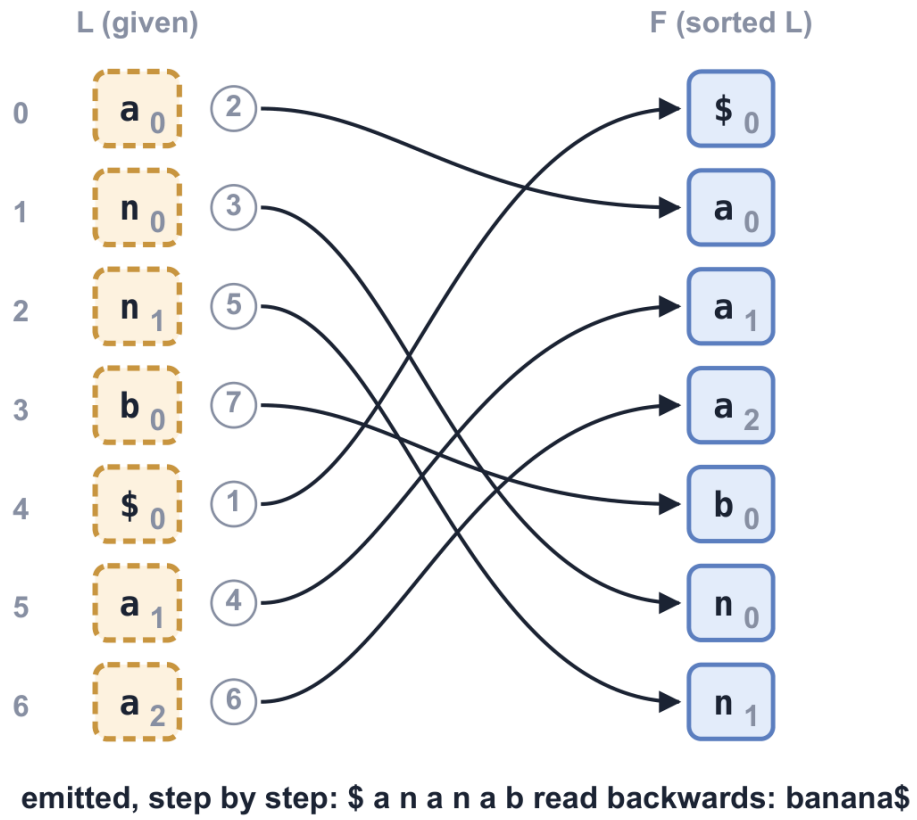
- Numeriraj kopije istog slova odozgo, od 0: u L su a_0, a_1, a_2 u retcima 0, 5, 6; u F u retcima 1, 2, 3.
- Tvrdnja: **a_2 u L je isti znak teksta kao a_2 u F** (vrijedi za svako slovo i svaki broj).
- Zašto: makni a — oba stupca sadrže iste nizove ($banan, na$ban, nana$b) u istom sortiranom poretku.

LF-preslikavanje

- **LF** (last-to-first): iz retka koji u L završava s $*c*_k$ skoči u redak koji u F počinje s $*c*_k$.
- Novi redak počinje **jednu poziciju ranije** u tekstu → LF hoda tekstom unatrag.
- **C[c]** = broj znakova u tekstu manjih od *c* (gdje u F počinje blok slova *c*). Za banana$: C[$]=0, C[a]=1, C[b]=4, C[n]=5.
- **occ(c, i)** = broj slova *c* u L iznad retka *i*.
- `LF(i) = C[c] + occ(c, i)`, gdje je `c = L[i]`; vrijedi `sa[LF(i)] = sa[i] - 1`.

Čitanje unatrag: vraćanje teksta iz L

- Kreni od retka koji u L ima $ (redak 4 = cijeli tekst).
- Ponavljaj *n* puta: ispiši L[redak], skoči redak = LF(redak).
- Redci 4 → 0 → 1 → 5 → 2 → 6 → 3 ispisuju $ a n a n a b → obrnuto: banana$.
- Samo iz L i tablice C — tekst nismo spremili.



Zašto $ mora biti jedinstven

- Bez jedinstvenog kraja ne znamo koji je redak cijeli tekst.
- Rotacije se tada ne sortiraju kao sufiksi: `abaa` → rotacije `2 3 0 1`, sufiksi `3 2 0 1`.
- Knjižnica odbija tekst s drugim $.

BWT nije kompresija (ali joj pomaže)

- Ista duljina, isti znakovi → ništa ne štedi: **1×**.
- Ali poredak skuplja slova sa sličnim kontekstom: ispred he u engleskom gotovo uvijek stoji `t`.
- *Moby-Dick*: od 14 623 pojavljivanja he, 12 455 ima `t` ispred → dugi nizovi `tttttttt` u L.
- Takve nizove lako je sažeti (bzip2) — o tome u poglavlju o kompresiji.
- Treća nadogradnja: **5× → 1×** — i SA nam više ne treba.

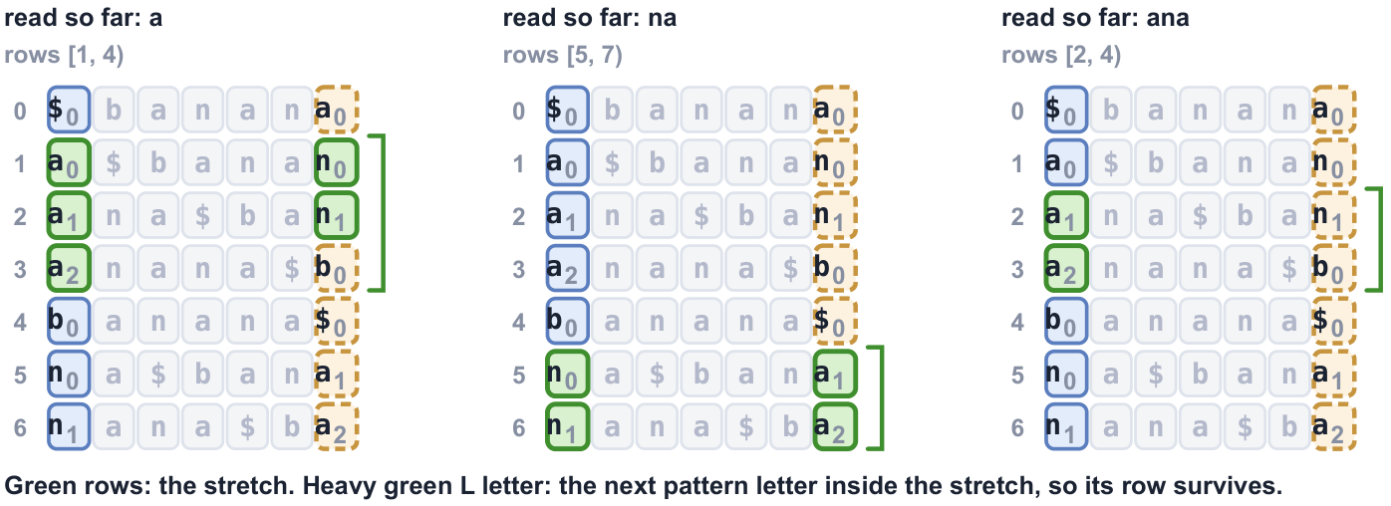
Odjeljak 5 — FM-indeks: knjiga napamet

Pitanje: možemo li baciti tekst?

- Servis koji odgovara svima drži *sve* knjige, scenarije i stihove. Indeks od 5× je najveći trošak.
- Što ako čuvamo **samo L** (veličine teksta) i bacimo tekst?
- Zadatak: imaš samo annb$aa i C. Pojavljuje li se ana i koliko puta?
- Savjet: koji retci sortirane matrice počinju **zadnjim** slovom uzorka?

Unatražna pretraga: intuicija

- Korak 1: retci koji počinju s a = blok slova a u F → raspon **[1, 4)**.
- Korak 2: retci koji počinju s na? Makni n → ostatak počinje s a → to je jedan od redaka 1–3.
- Pitanje postaje: koji od redaka 1–3 ima n **neposredno ispred** sebe? To je zadnje slovo — u L!
- Retci 1 i 2 završavaju s n_0 i n_1 → LF ih vodi u retke 5 i 6 (na$bana, nana$ba).
- **Invarijanta:** raspon su točno retci koji počinju dosad pročitanim dijelom uzorka.



Od intuicije do formule: dva brojanja po slovu

- Brojevi uz n u L su njihovo mjesto u bloku slova n u F.
- Broj prvog n unutar raspona = koliko n ima u L **iznad** početka raspona.
- Za raspon [1, 4): occ(n, 1) = 0, occ(n, 4) = 2 → novi raspon [5 + 0, 5 + 2) = **[5, 7)**.
- Općenito, čitanje slova *c* ulijevo:

```
[lo, hi) → [ C[c] + occ(c, lo),  C[c] + occ(c, hi) )
```

Razrađeni primjer: ana u banana$

čitamo	lo	hi	retci	počinju s
a	1	4	1, 2, 3	a
n	5 + 0 = 5	5 + 2 = 7	5, 6	na
a	1 + 1 = 2	1 + 3 = 4	2, 3	ana

- `count(ana)` = 4 − 2 = **2**. Tekst nismo ni pogledali.

Trošak: ne ovisi o duljini teksta

- Svako slovo uzorka = **dva brojanja** occ(c, lo) i occ(c, hi).
- Uzorak od 24 slova = 48 pitanja — bilo da je tekst `banana$` ili cijela knjižnica.
- Sve sad visi o jednoj operaciji: **koliko slova *c* ima iznad retka *i*** (to je *rank*).
- Preostala tri posla: brzo brojanje u malo prostora, `locate`, `extract`.

Brzo brojanje: kontrolne točke

- Brojanje od početka L svaki put = ponovno skeniranje. Ne!
- Svakih *k* redaka spremi koliko je bilo svakog slova do tog retka.
- `occ(c, i) = occ(c, kontrolna točka) + broj slova c između`.
- Primjer, *k* = 4: točka u retku 4 pamti annb → a:1, b:1, n:2. occ(a, 7) = 1 + 2 = **3**.
- U praksi: L se sprema komprimirano (wavelet stablo) — "crna kutija" iz poglavlja o kompresiji.

`locate`: gdje je pogodak? Uzorkovanje

- Retci raspona su retci SA — ali SA (4n) smo upravo bacili.
- Rješenje: spremi poziciju samo za svaku 32. poziciju teksta i označi te retke.
- Za neoznačeni redak: hodaj LF-om unatrag dok ne dođeš do označenog; **pozicija = spremljena + broj koraka**.
- Primjer (stopa 2, spremljene pozicije 0, 2, 4, 6): redak 2 → (n_1) LF → redak 6 = poz. 2 → **3**. Redak 3 → (b_o) LF → redak 4 = poz. 0 → **1**.



row	rotation	SA	LCP	F	L	kept
0	$banana	6	0	$	a_0	●
1	a$banan	5	0	a	n_0	
2	ana$ban	3	1	a	n_1	
3	anana$b	1	3	a	b_0	
4	banana$	0	0	b	$\$_0$	●
5	na$bana	4	0	n	a_1	●
6	nana$ba	2	2	n	a_2	●

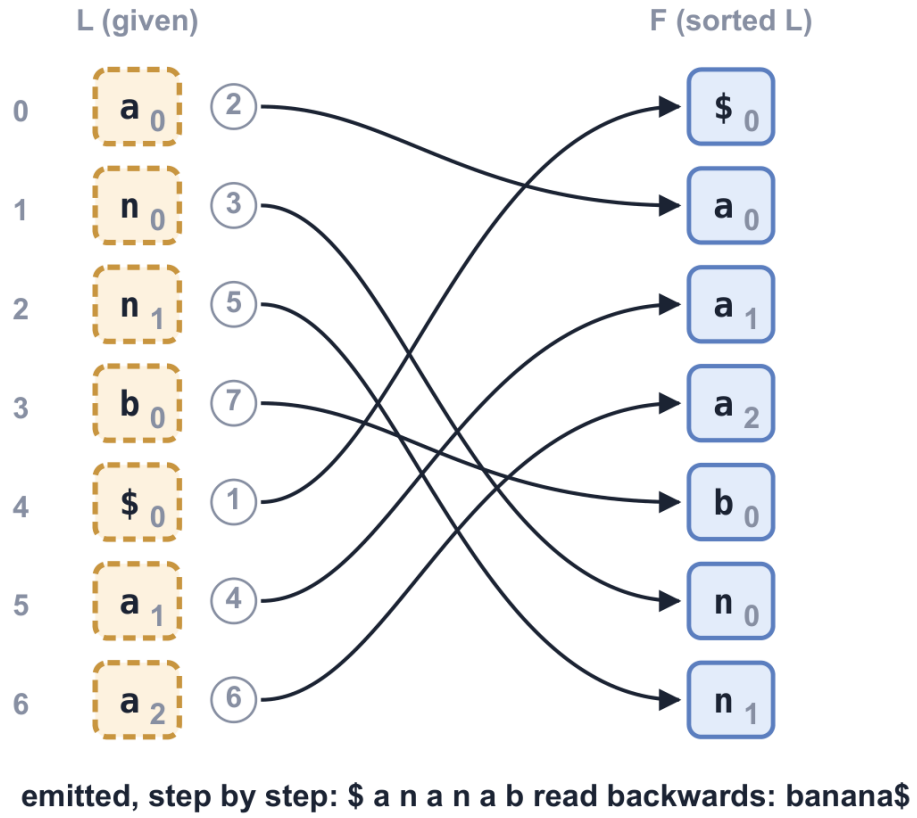
Stopa uzorkovanja: kotačić prostor ↔ vrijeme

uzorak svakih	veličina indeksa	vrijeme po poziciji
16 pozicija	0,882 n	1,2 µs
32 pozicije	0,685 n	2,4 µs
64 pozicije	0,586 n	4,6 µs

- Dvostruko rjeđe uzorkovanje → dvostruko dulji hod, a ušteda prostora se svaki put prepolovi.

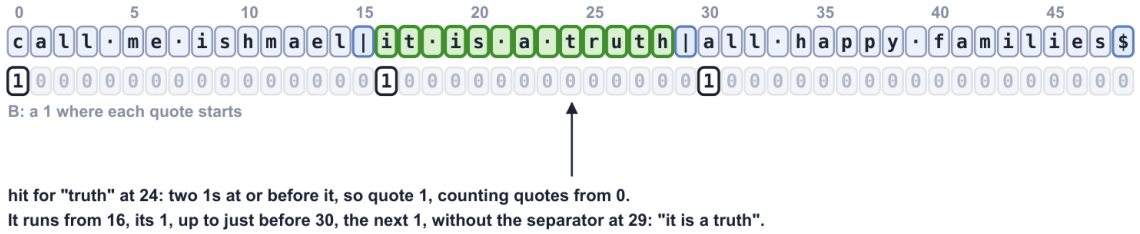
`extract`: knjiga iz indeksa

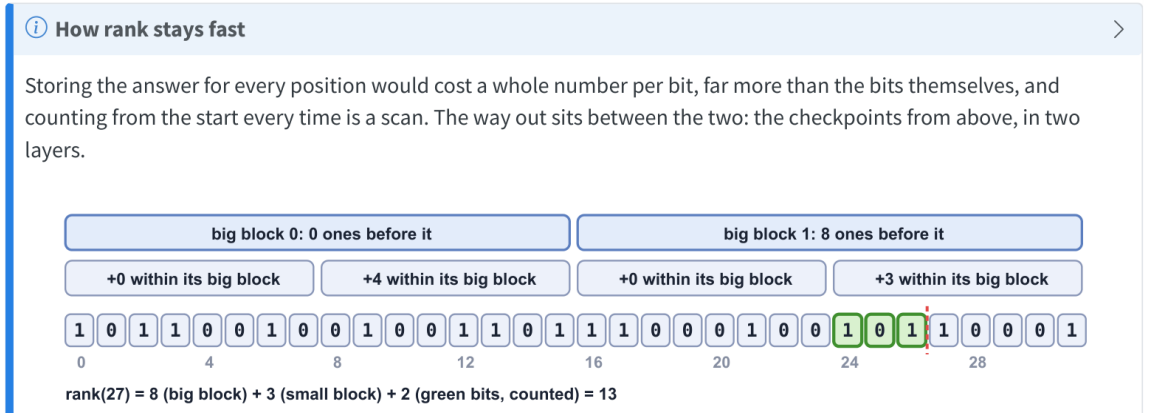
- Tekst je bačen — `extract` se mora napraviti hodanjem.
- Druga lista uzoraka: redak za svaku 32. poziciju.
- Za raspon [s, e]: kreni od prvog spremljenog *t* > e, hodaj LF *t* − *s* koraka, obrni, zadrži prvih ℓ.
- Primjer: banana$, s = 1, ℓ = 2 → t = 4 (redak 5) → retci 5, 2, 6 → a n a → obrnuto ana → **an**.
- Trošak: ~ℓ + stopa koraka. Svaki redak knjige može se pročitati iz indeksa.



Od pogotka do cijelog citata: rank i select

- Svi citati spojeni u jedan tekst, separator između, $ na kraju.
- Bitovni niz **B**: 1 tamo gdje citat počinje.
- Za pogodak na poziciji *p*: `citat = rank(p + 1) − 1`, `početak = select(citat)`, `separator = select(citat + 1) − 1`.
- Primjer: `truth` na poziciji 24 → citat 1 → pozicije 16–28 → "it is a truth".
- rank u konstantnom vremenu: dvorazinski direktorij (veliki + mali blokovi).





Koliko košta i gdje puca

- Deset knjiga: **0,685 bajta po bajtu teksta** (4,66 MB naspram 6,79 MB) — i tekst nam ne treba.
- 7,3× manje od teksta + SA. Brojanje i lociranje: nekoliko mikrosekundi.
- Cijena: ~3× sporije brojanje i ~6× sporije lociranje od nekomprimiranog FM-indeksa (~2n).
- U praksi: 83 TB web teksta → indeks 44 % veličine teksta (2025.); aligneri DNA BWA i Bowtie.
- Slabost: **zamrznut je** — novi dokument mijenja retke posvuda → gradi ispočetka.

Pogled unutra: od čega je sve građeno

- Sufiksno polje = pozicija umjesto kopije + udvostručavanje + podijeli pa vladaj
- BWT = sufiksno polje + jednaka slova čuvaju poredak
- FM-indeks = BWT + rank/select + rad u blokovima + uzorkuj pa hodaj
- Citat iz pogotka = FM-indeks + rank/select + rezervirani separatori



Odjeljak 6 — Usporedba i zaključak

Usporedba

struktura	priprema	count	locate	veličina
Skeniranje (Rabin–Karp)	ništa	~n + m	isto	tekst, n
Patricia stablo (k ključeva)	~ukupna duljina	m koraka	+ korak po odgovoru	≤ 2k čvorova + ključevi
Sufiksno polje	~n log n	~2m $\log_2$ n	+ z	5n (s tekstom)
FM-indeks	SA, jednom	2m brojanja	+ ≤ stopa po pogotku	0,685 n, bez teksta

Kad koristiti što

- Tekst se pretražuje jednom ili se stalno mijenja → **skeniranje**.
- Znam početak (autocomplete, usmjerivači paketa) → **trie / Patricia**.
- Bilo koji komadić, tekst stane više puta u memoriju → **sufiksno polje**.
- Velik, nepromjenjiv tekst, prostor je bitniji od mikrosekundi → **FM-indeks**.
- Odgovor mora biti cijeli zapis → **FM-indeks + rank/select**.

Pitanja za vježbu

1. 100 GB nepromjenjivog teksta, 128 GB memorije, milijuni upita dnevno. Kolega predlaže sufiksno polje. Koliko memorije treba i što biste izabrali umjesto toga? Što postaje sporije i koji parametar odlučuje koliko?
2. Isti servis dobiva tisuću novih dokumenata dnevno, pretraživih unutar sat vremena. Koju pretpostavku FM-indeksa to ruši i kako to riješiti bez stalne ponovne izgradnje?
3. 10 000 genoma od po 3 milijarde slova, međusobno različitih u jednom slovu od tisuću. Zašto je "veličina jednog FM-indeksa × 10 000" i dalje previše? (Savjet: nizovi u transformaciji.)